Ecommerce Batch Data Pipeline

Technical Project Documentation

Repository: https://github.com/shoman042/ecommerce-etl-pipeline

1. Introduction

This document describes the design, implementation, and validation of an end-to-end batch data pipeline built for synthetic e-commerce data. The pipeline covers data generation, storage, extraction, transformation, and analytical modeling, following a layered architecture that spans a relational source system, a distributed file system, a distributed processing engine, and a data warehouse layer.

2. Project Overview

The project implements a batch ETL (Extract, Transform, Load) pipeline for e-commerce data. Synthetic data is generated using Python and stored in a MariaDB relational database. The data is then extracted using Apache NiFi and stored as raw data in the Hadoop Distributed File System (HDFS). Apache Spark is used to clean and transform the raw data into analytics-ready datasets, which are subsequently exposed as external tables in Apache Hive for analytical querying.

The overall data flow follows this sequence:

- Python Data Generator
- MariaDB (Source Tables)
- Apache NiFi (Extraction)
- HDFS (Raw Layer / Staging Zone)
- Apache Spark (Transformation)
- Apache Hive (Analytics Tables)

3. Objectives

- Generate realistic synthetic e-commerce data using Python.
- Store the generated data in a MariaDB relational database using batch insertion for performance.
- Extract the source data from MariaDB into HDFS using Apache NiFi, preserving the raw, untransformed data.

- Clean and validate the raw data using Apache Spark, including handling of duplicates, null values, data type conversions, and validation of dates, prices, and quantities.
- Apply business transformations to produce analytics-ready datasets covering sales performance by customer, product, country, category, and time period.
- Load the transformed datasets into Apache Hive as a queryable data warehouse layer.
- Validate data quality and consistency across all pipeline layers through record-count and reconciliation checks.

4. Scope

The scope of the project covers the full batch pipeline from data generation through to the creation of analytics tables in Hive. It includes the design and configuration of the NiFi extraction flow, the raw data (staging) layer in HDFS, the Spark transformation logic, and the corresponding Hive data warehouse schema. Real-time or streaming processing is outside the scope of this project, as the pipeline is designed and executed as a batch process.

5. System Architecture

The pipeline follows a layered architecture in which data moves sequentially through a source system, a raw storage layer, a processing layer, and an analytics layer. The architecture is as follows:

- Python Generator: produces synthetic e-commerce records and inserts them into MariaDB.
- MariaDB: acts as the source system of record, holding the source tables.
- Apache NiFi: extracts data from MariaDB and writes it, without business transformation, into HDFS.
- HDFS Raw Layer: stores the extracted data in a staging zone, organized by source table.
- Apache Spark: reads the raw layer, applies data cleaning and business transformations, and writes analytics-ready datasets back to HDFS.
- Apache Hive: exposes the transformed datasets as external tables within a dedicated data warehouse database for analytical querying.

6. Technologies and Tools

- Python — synthetic data generation, using the Faker library and the mysql-connector-python driver.
- MariaDB — relational database used as the system of record for source data.
- Apache NiFi (version 1.14.0) — data extraction from MariaDB into HDFS.
- Apache Hadoop / HDFS — distributed storage for the raw and analytics data layers.
- Apache Spark (PySpark) — distributed data cleaning and transformation.

- Apache Hive (version 3.1.2) — data warehouse layer for analytical tables.
- MariaDB Java Client (mariadb-java-client-3.5.10.jar) — JDBC driver used by NiFi to connect to MariaDB.

7. Data Generation

A Python script generates synthetic e-commerce data and inserts it directly into MariaDB. The script creates the required database and tables, and populates them using batch insertion (in batches of 5,000 records) rather than single-row inserts, in order to improve performance for large volumes of data.

The following data volumes were generated:

- Customers: 100,000 records
- Products: 20,000 records
- Orders: 500,000 records
- Order Items: 2,000,000 records
- Payments: 500,000 records

Customer records include personal and geographic attributes such as name, email, country, city, and signup date. Product records include product name, category, price, and stock quantity. Orders include the associated customer, order date, status, and shipping country. Order items link orders to products with quantity and unit price. Payments record the payment method, status, date, and amount associated with each order.

8. Data Storage in MariaDB

MariaDB hosts the source database, named “ecommerce,” containing five tables: customers, products, orders, order_items, and payments. Tables are created programmatically by the data-generation script, which first drops any existing tables before recreating them, ensuring a consistent starting state for each run.

9. Data Extraction Using Apache NiFi

Apache NiFi is used to extract the source tables from MariaDB and store them, without applying business transformations, in the HDFS raw layer. The extraction flow for each table follows the same processor sequence:

- ExecuteSQL: executes a SELECT * query against the source table using a DBCPConnectionPool controller service configured with the MariaDB JDBC connection.
- ConvertRecord: converts the query result from its embedded Avro schema into CSV format using an AvroReader and a CSVRecordSetWriter, with no header line, to align with the format expected by the downstream Spark process.

- PutHDFS: writes the resulting CSV data to a table-specific directory under the HDFS staging zone, using a replace conflict-resolution strategy.

The DBCPConnectionPool controller service is configured with the MariaDB JDBC URL, the org.mariadb.jdbc.Driver driver class, and the mariadb-java-client-3.5.10.jar driver library. Date and timestamp fields are explicitly formatted within the CSVRecordSetWriter to ensure that they are written as readable date and time strings rather than raw numeric (epoch) values.

The same processor sequence is applied independently to each of the five source tables: customers, products, orders, order_items, and payments.

10. Raw Data Storage in HDFS

The raw data extracted by NiFi is stored under a staging zone directory structure in HDFS, organized by source table, at the path /staging_zone/ecommerce/. Each table is stored in its own subdirectory (for example, /staging_zone/ecommerce/customers), preserving the original source data without business transformations.

Following extraction, the row counts in HDFS were verified against the corresponding MariaDB source tables and found to match exactly:

- Customers: 100,000 records
- Products: 20,000 records
- Orders: 500,000 records
- Order Items: 2,000,000 records
- Payments: 500,000 records

11. Data Transformation Using Apache Spark

A PySpark application reads the raw data from the HDFS staging zone using explicit schemas, since the extracted CSV files do not contain header rows. The application performs data cleaning followed by business transformations.

11.1 Data Cleaning

- Removal of duplicate records based on each table's primary key.
- Handling of null values, including filtering of records with missing critical identifiers and filling of non-critical missing text fields with default values.
- Conversion of columns to appropriate data types, including explicit parsing of date and timestamp fields.
- Validation of dates, excluding records with missing or future-dated values.
- Validation of prices and quantities, excluding records with zero or negative values.

11.2 Business Transformations

A unified sales dataset is created by joining the order_items, orders, products, and customers tables. From this joined dataset, the following metrics are calculated:

- Total sales
- Total quantity sold
- Average order value
- Number of orders
- Number of customers
- Sales by country
- Sales by product category
- Sales by day
- Sales by month
- Sales by year

12. Analytics Tables

The Spark application produces the following analytics-ready datasets, written to HDFS in Parquet format:

12.1 Fact Sales

Contains one record per order item, including order_id, customer_id, product_id, order_date, quantity, unit_price, total_amount, category, country, and status. This table is partitioned by country.

12.2 Daily Sales

Aggregates sales by year, month, and day, including total_orders, total_quantity, total_sales, and average_order_value.

12.3 Customer Sales

Aggregates sales by customer, including customer_id, customer_name, country, total_orders, total_quantity, and total_spending.

12.4 Product Sales

Aggregates sales by product, including product_id, product_name, category, total_quantity, and total_sales.

12.5 Additional Rollups

In addition to the four primary analytics tables, the following supplementary rollup datasets were produced to support the required business calculations:

- Monthly Sales — aggregated by year and month.
- Yearly Sales — aggregated by year.
- Sales by Country — aggregated totals per country.
- Sales by Category — aggregated totals per product category.

13. Data Warehouse in Apache Hive

A dedicated Hive database, ecommerce_dw, was created to host the analytics layer. Each analytics dataset produced by Spark is exposed as an external Hive table, pointing directly to its corresponding Parquet location in HDFS. The following external tables were created: fact_sales, daily_sales, customer_sales, product_sales, monthly_sales, yearly_sales, sales_by_country, and sales_by_category.

14. Testing and Validation

Data quality and consistency were validated at multiple stages of the pipeline through dedicated validation queries executed separately against MariaDB and Hive.

14.1 Source Layer Validation

Record counts in the MariaDB source tables were confirmed to match the volumes generated by the Python data-generation script (100,000 customers, 20,000 products, 500,000 orders, 2,000,000 order items, and 500,000 payments). A referential integrity check confirmed zero orphaned orders (orders referencing a non-existent customer).

14.2 Analytics Layer Validation

The analytics tables in Hive were validated for row counts, null values in key columns, invalid (zero or negative) prices and quantities, correctness of the total_amount calculation, absence of future-dated records, reconciliation of aggregate totals across the fact_sales, daily_sales, customer_sales, and product_sales tables, and absence of duplicate records at the order_id and product_id grain.

During validation, product_sales was observed to contain 20,000 rows, matching the full product catalog, while customer_sales contained approximately 99,255 rows out of 100,000 total customers. This difference is expected, as customer_sales includes only customers with at least one associated order, and a small number of customers did not receive any orders given the random distribution of 500,000 orders across 100,000 customers.

15. Challenges and Solutions

- Extraction tool selection: the project requirements specified Apache NiFi for extraction. An initial extraction approach using Sqoop was replaced with a dedicated NiFi flow (ExecuteSQL, ConvertRecord, PutHDFS) to align with the required architecture.
- Date and timestamp formatting: NiFi's ConvertRecord processor initially risked writing date and timestamp values as raw numeric (epoch) values. This was resolved by explicitly configuring Date and Timestamp Format properties in the CSVRecordSetWriter.
- Header-less raw files: since the raw CSV files extracted by NiFi do not include header rows, the Spark application was configured to read the data using explicit, predefined schemas matching the source table structures.
- Validation query organization: running combined validation queries directly against Hive initially failed when queries intended for MariaDB were included in the same script. This was resolved by separating the source-layer (MariaDB) and analytics-layer (Hive) validation queries into distinct scripts, and running Hive in silent mode to produce clean, readable query output.

16. Deliverables

All source code, configuration scripts, and validation queries for this project are available in the project repository at: https://github.com/shoman042/ecommerce-etl-pipeline

- Python data-generation script.
- MariaDB database and table creation logic.
- Apache NiFi extraction flow.
- HDFS raw-data (staging zone) directory structure.
- PySpark transformation script.
- Hive database and table creation script.
- Data-quality and record-count validation queries, for both the MariaDB source layer and the Hive analytics layer.

17. Conclusion

The project successfully implements a complete batch data pipeline for synthetic e-commerce data, spanning data generation, relational storage, extraction, distributed raw storage, transformation, and analytical warehousing. Validation checks performed across the source and analytics layers confirm that data volumes and business calculations are consistent throughout the pipeline, demonstrating the reliability of the end-to-end process.